

DC Lab — Experiment 4 Walkthrough

Ricart-Agrawala Mutual Exclusion over gRPC

(Mini Project: ArtistAlley — prevents multiple simultaneous submissions)

Goal: Three equal peer processes (Artist nodes) compete for access to one shared resource — a “Featured Submission” slot. Only one may hold it at a time, with no central coordinator. Coordination happens entirely through gRPC calls between the peers, using Lamport timestamps to decide priority.

Prerequisite: Exp 2 and Exp 3 already done — you need an existing `proto/artwork.proto`, an activated `venv` with `grpcio + grpcio-tools` installed, and a working understanding of Lamport clocks (tick before send, `max(local, received) + 1` on receive).

Step 1 — Add MutexService to the proto file

Append this to your existing `proto/artwork.proto` (don’t create a new file — one proto file per project is cleaner):

```
// Peer-to-peer mutual exclusion service.
// Every Artist node runs this server AND calls it on its peers.
service MutexService {
    rpc RequestAccess (AccessRequest) returns (AccessReply);
}

message AccessRequest {
    int32 node_id = 1;
    int32 timestamp = 2;
}

message AccessReply {
    int32 node_id = 1;
    int32 timestamp = 2;
}
```

Step 2 — Regenerate the stubs

```
python -m grpc_tools.protoc -I=proto --python_out=. --grpc_python_out=. proto/art
```

Verify both services compiled in:

```
grep -E "class (ArtworkService|MutexService)" artwork_pb2_grpc.py
```

You should see 3 classes for each service (`Stub` , `Servicer` , and the base class).

Step 3 — Understand the core trick before you write code

In a normal client-server RPC, the server always replies right away. Ricart-Agrawala needs the option to **defer** a reply until later. The trick: a gRPC handler doesn't have to `return` immediately — it can block internally, and the RPC only completes once it finally returns. So:

- **"Grant a reply"** = the handler returns right away
- **"Defer a reply"** = the handler blocks (e.g. waits on a `threading.Event`) until it's that peer's turn, *then* returns

No separate "REPLY message" is needed — the RPC response itself is the reply.

Step 4 — Write `node.py`

Each of the 3 terminals will run the exact same file, just with a different node ID argument (`python node.py 1` , `2` , or `3`). Structure:

```
import sys, time, threading
from concurrent import futures
import grpc
import artwork_pb2, artwork_pb2_grpc

PEERS = {1: "localhost:60051", 2: "localhost:60052", 3: "localhost:60053"}

class Node:
    def __init__(self, node_id):
        self.id = node_id
        self.peers = {nid: addr for nid, addr in PEERS.items() if nid != node_id}
        self.clock = 0
        self.state = "RELEASED"          # RELEASED -> WANTED -> HELD
        self.request_time = None
        self.lock = threading.Lock()
        self.deferred = []               # threading.Events for peers we're making
```

```

def tick(self):
    with self.lock:
        self.clock += 1
    return self.clock

def update(self, received_ts):
    with self.lock:
        self.clock = max(self.clock, received_ts) + 1
    return self.clock

def serve(self):
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    artwork_pb2_grpc.add_MutexServiceServicer_to_server(_Servicer(self), serv
    server.add_insecure_port(PEERS[self.id])
    server.start()
    return server

def _call_with_retry(self, addr, my_ts, max_wait=30):
    # Peers may not be up yet – retry instead of crashing
    deadline = time.time() + max_wait
    while time.time() < deadline:
        try:
            with grpc.insecure_channel(addr) as channel:
                stub = artwork_pb2_grpc.MutexServiceStub(channel)
                return stub.RequestAccess(
                    artwork_pb2.AccessRequest(node_id=self.id, timestamp=my_t
        except grpc.RpcError:
            time.sleep(0.5)
    raise RuntimeError(f"Could not reach {addr}")

def request_cs(self):
    my_ts = self.tick()
    with self.lock:
        self.state = "WANTED"
        self.request_time = my_ts

    for peer_id, addr in self.peers.items():
        reply = self._call_with_retry(addr, my_ts)
        self.update(reply.timestamp)

    with self.lock:
        self.state = "HELD"
    print(f"Node-{self.id}: ENTERED critical section")
    time.sleep(0.5)          # simulate doing the submission
    self.release_cs()

def release_cs(self):

```

```

        with self.lock:
            self.state = "RELEASED"
            to_release, self.deferred = self.deferred, []
        for event in to_release:
            event.set()                # let deferred peers' RPCs finally return

class _Servicer(artwork_pb2_grpc.MutexServiceServicer):
    def __init__(self, node):
        self.node = node

    def RequestAccess(self, request, context):
        node = self.node
        node.update(request.timestamp)

        with node.lock:
            defer = (
                node.state == "HELD"
                or (node.state == "WANTED"
                    and (node.request_time, node.id) < (request.timestamp, request.id)
                )

            )

        if defer:
            event = threading.Event()
            with node.lock:
                node.deferred.append(event)
            event.wait()                # blocks the RPC response — this IS the decision logic

        send_ts = node.tick()
        return artwork_pb2.AccessReply(node_id=node.id, timestamp=send_ts)

def main():
    node = Node(int(sys.argv[1]))
    server = node.serve()
    time.sleep(6)                      # let all 3 nodes finish starting up first
    node.request_cs()
    time.sleep(2)
    server.stop(0)

if __name__ == "__main__":
    main()

```

The decision logic in `RequestAccess` is the entire algorithm — everything else is plumbing. Read it carefully:

- Defer if you're currently **holding** the critical section, OR
- Defer if you **also want it** and your own `(timestamp, node_id)` pair is smaller than theirs (i.e. you have priority and should go first)
- Otherwise, grant immediately

Step 5 — Run all 3 peers

Open 3 terminal tabs, `cd` into the project folder and activate `venv` in each, then start them close together (within a couple seconds of each other):

```
# Terminal 1
python node.py 1

# Terminal 2
python node.py 2

# Terminal 3
python node.py 3
```

The 6-second startup delay gives you breathing room to switch tabs. Even if you're not perfectly in sync, the retry logic means no one crashes — a node just quietly retries until its peer is reachable.

Step 6 — What to check in the output

- Every node should print `WANTS`, then either `GRANTS` / `DEFERS` for others' requests, then `ENTERED critical section`
- **No two nodes should ever be inside the critical section at the same time**
- Requests should be honored in ascending timestamp order — the node with the lowest `(timestamp, node_id)` among currently-competing requests always goes first
- If you want to actually see a `DEFERS` in the logs (proving real contention), try starting all 3 terminals as fast as possible — within ~1 second of each other — so their requests genuinely overlap

Step 7 — Commit and push

```
git add proto/artwork.proto node.py artwork_pb2.py artwork_pb2_grpc.py
git commit -m "DC Lab Exp 4: Ricart-Agrawala mutual exclusion over gRPC"
```

```
git push origin main
```

Common mistakes

Mistake	Symptom	Fix
Forgetting to activate <code>venv</code> in a new tab	<code>ModuleNotFoundError: No module named 'grpc_tools'</code>	<code>source venv/bin/activate</code> in every tab before running
Starting nodes with big gaps between them	<code>Connection refused</code> errors, node crashes	Use the retry helper; start peers within a few seconds of each other
Replying immediately every time	Never see a <code>DEFERS</code> — no real mutual exclusion demonstrated	Only grant instantly when the state/priority check says so
Using a separate <code>REPLY</code> message	Overcomplicates the code	Let the RPC response itself act as the reply — that's the point of blocking the handler